

Benchmark Results and Comparative Evaluation

Measured execution classification, latency, diagnostic-evidence coverage, and current continuous-integration stability

T. R. Bentley

Empirical Results Paper | Paper 8

September 2026

Revision note. This version adds a newly executed frozen 24-case benchmark against the current executor implementation from repository commit 6bc57eb0f37b9acbdba4f376e26f13610eccac7a. All four language toolchains were available in the benchmark environment. The benchmark is an execution/failure-classification experiment, not an LLM repair-accuracy experiment.

Current CI evidence: 27/27 tests passed on Python 3.10 and Python 3.12. Frozen benchmark raw task-level data are supplied as a companion CSV.

Abstract

This paper reports measured empirical results for AI Debugger Pro using two evidence layers. First, a newly executed frozen benchmark exercised 24 representative programs across Python, C, C++, and Java, including successful execution, syntax or compilation failure, runtime failure, and deliberate timeout behavior. All 24 expected success/failure classifications were matched (24/24, 100% observed classification accuracy). Overall median measured latency was 282.969 ms and mean latency was 351.798 ms. Second, the current GitHub Actions baseline passed 27/27 repository tests under both Python 3.10 and Python 3.12, with Ruff passing in both jobs.

The benchmark also identified a diagnostic-evidence limitation: 14 of 16 observed failures returned non-empty textual evidence, while two native runtime failures - one C and one C++ process terminating with a nonzero exit status but no stderr - were correctly classified as failures with empty diagnostic text. The result shows that failure detection and evidence richness are distinct system properties. The experiment supports claims about the tested execution layer; it does not yet measure LLM-generated repair correctness or competitive superiority.

1. Experimental Design

The frozen corpus contains 24 cases: eight Python, six C, six C++, and four Java. Cases were fixed before scoring and span four operational classes: success, syntax/compile failure, runtime failure, and timeout. Expected success/failure status served as the classification oracle. The implementation under test was reconstructed verbatim from `core/executor.py` at commit `6bc57eb`. The environment provided Python, gcc, g++, javac, and java. Normal cases used a 3 s ceiling; deliberate timeout cases used 0.25 s.

Language	Cases	Matches	Accuracy	Median ms
C	6	6	100%	36.397
CPP	6	6	100%	161.672
JAVA	4	4	100%	399.947
PYTHON	8	8	100%	754.159

2. Frozen Benchmark Results

Every frozen task produced the expected binary execution classification. This means the execution layer distinguished the selected successful and failing programs correctly across all four available language toolchains in this corpus.

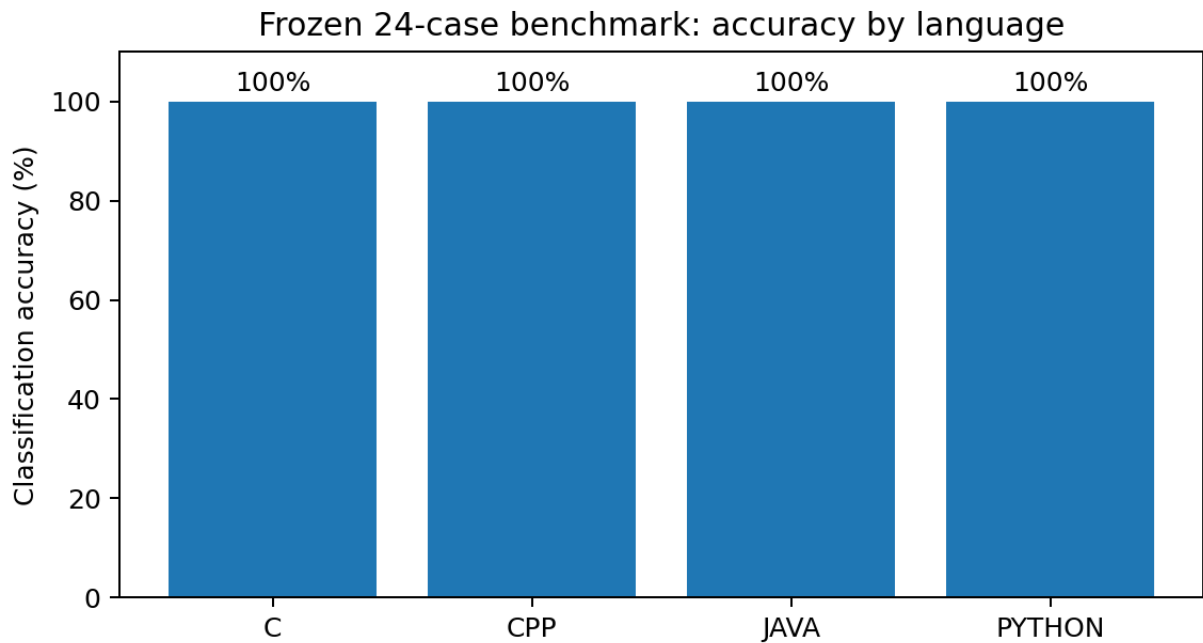


Figure 1. Observed success/failure classification accuracy by language in the newly executed frozen benchmark. Each language reached 100% on its selected cases; this small controlled corpus does not imply universal 100% debugging accuracy.

3. Latency

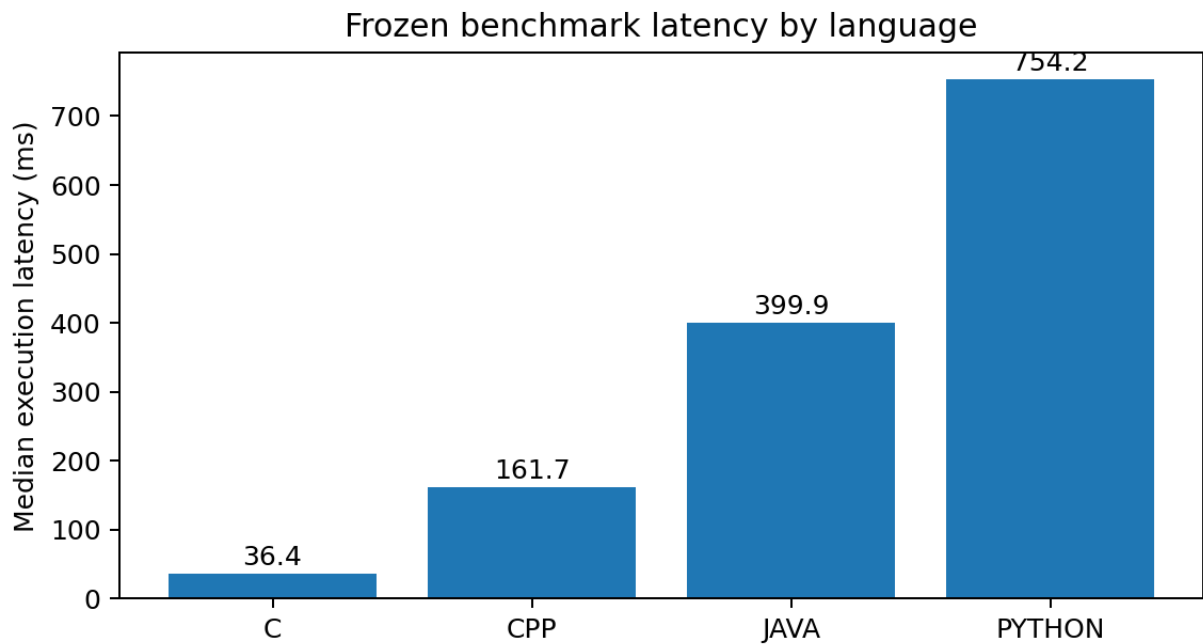


Figure 2. Median end-to-end execution latency by language. Overall median was 282.969 ms; overall mean was 351.798 ms. Values include process startup, compilation where applicable, execution, capture, and deliberate timeout cases.

4. Diagnostic Evidence Coverage

Sixteen benchmark tasks produced an observed failure. Fourteen (87.5%) returned non-empty textual diagnostic evidence. Two (12.5%) returned an empty diagnostic string despite correct failure classification.

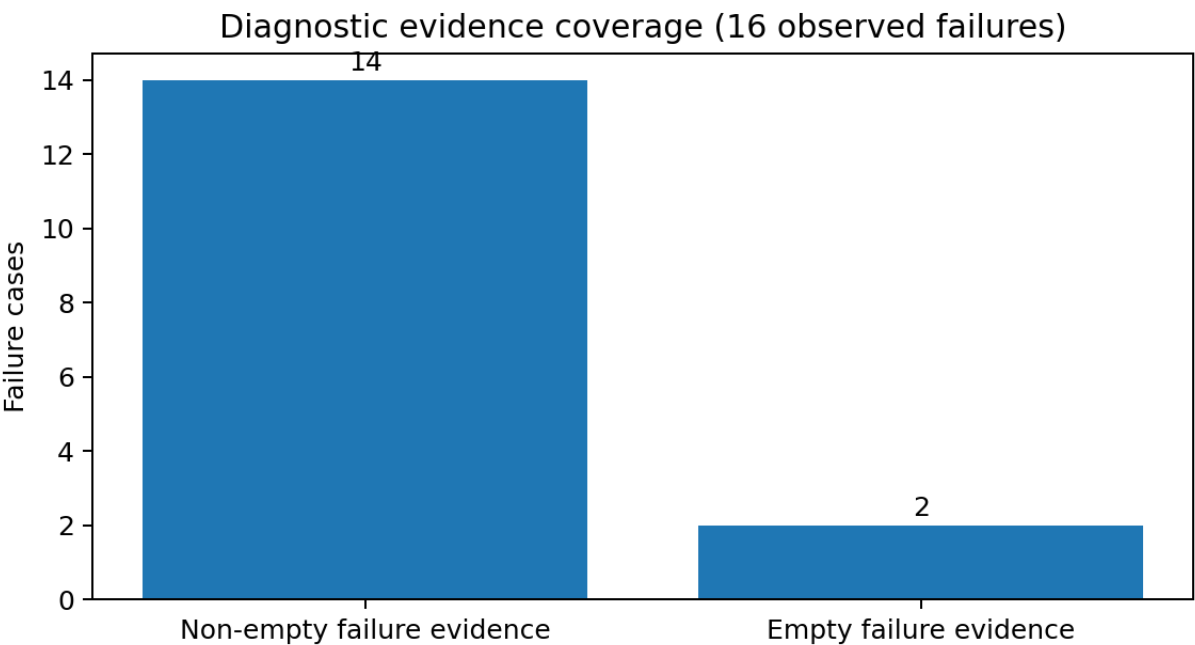


Figure 3. Textual evidence coverage among observed failures. The empty-evidence cases were C03 and CPP03: native programs that deliberately returned nonzero status without emitting stderr.

5. Current Repository CI

Environment	Tests passed	Failures	pytest runtime	Ruff
Python 3.10.21	27	0	1.12 s	Pass
Python 3.12.14	27	0	1.17 s	Pass

The CI results establish regression stability for the repository's existing automated test surface at the measured commit. They are complementary to, not substitutes for, the frozen behavioral benchmark.

6. Interpretation

The new benchmark strengthens Paper 8 in two ways. First, the paper now contains a fresh task-level experiment executed specifically for the empirical evaluation rather than relying only on repository CI or an earlier pilot. Second, the raw data expose a system limitation instead of merely producing a pass rate: nonzero native process termination is detectable, but stderr-only diagnostic extraction can yield no explanatory text.

The appropriate architectural response is to represent termination evidence explicitly - exit code, signal where available, timeout status, compilation phase, stdout/stderr presence, language/toolchain, and backend - rather than treating a single output string as the entire failure record. This finding can directly inform a future executor contract revision.

7. Claim Boundary

Supported by these measurements	Still requires a separate experiment
24/24 selected frozen execution cases matched expected binary classification.	LLM repair pass@1 or semantic patch correctness.
All four configured language toolchains were exercised in this benchmark environment.	Comparative superiority to external AI debugging tools.
14/16 observed failures supplied non-empty textual evidence.	Human approval improves developer decisions.
27/27 current repository tests pass on both configured Python CI versions.	General 100% accuracy on arbitrary programs.
Latency is now measured at task level for this environment.	Universal performance or cross-machine speed claims.

8. Threats to Validity

The corpus is intentionally small and synthetic. Binary success/failure classification is weaker than semantic correctness. Language case counts are unequal. Latency is environment-dependent and combines interpreted execution, native compilation, JVM startup, and timeouts. The executor source was reconstructed verbatim from the connected repository because direct network cloning was unavailable in the benchmark runtime; therefore the experiment targets the executor module rather than an installed full-repository checkout. CI separately confirms the repository's full automated test suite at the same commit.

9. Next Experiment: Repair Accuracy

The next frozen corpus should test the AI repair loop itself. Each defective program needs a hidden behavioral oracle, a permitted patch scope, and labels for correct, overfit, suppressive, insecure, and abstention-worthy repairs. The primary measurements should be pass@1 semantic repair rate, regression-free rate, verification-catch rate, false-verification rate, abstention precision, confidence calibration, and patch size. That experiment would answer a different question from the one measured here: not whether the executor recognizes failure, but whether AI-generated source mutation actually repairs the defect without damaging intended behavior.

10. Conclusion

Paper 8 now contains a newly executed benchmark rather than only inherited pilot measurements. Across 24 frozen tasks, AI Debugger Pro's executor matched all expected binary outcomes. The measured overall median latency was 282.969 ms. Diagnostic text was non-empty for 14 of 16 failures, with two native nonzero-exit cases exposing an evidence gap. Together with a current 27/27 CI pass on both configured Python versions, these data provide a reproducible engineering baseline while preserving a clear boundary around claims that still require an LLM repair benchmark.

Data Availability

Two companion CSV files are supplied: task-level frozen benchmark observations and an aggregate measurement table. The task-level file records case ID, language, failure class, expected/observed status, match, latency, evidence presence, and bounded output.